

Part A — City Library Management System

Working Python Code

```
from collections import Counter
from typing import Dict, List

class Book:
    """
    Represents a book in the library.
    """

    def __init__(self, book_id: int, title: str, author: str, genre: str):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.genre = genre
        self.is_available = True # True = Available, False = Issued

    def __repr__(self):
        status = "Available" if self.is_available else "Issued"
        return f"[{self.book_id}] {self.title} by {self.author} ({self.genre}) - {status}"

class Member:
    """
    Represents a library member.
    """

    def __init__(self, member_id: int, name: str, age: int, contact: str):
        self.member_id = member_id
        self.name = name
        self.age = age
        self.contact = contact
        self.borrowed_books: List[int] = [] # List of book IDs

    def __repr__(self):
        return f"[{self.member_id}] {self.name}, Age: {self.age}, Contact: {self.contact}"

class Library:
    """
    Main Library Management System.
    Handles books, members, borrowing, returning, and reporting.
    """

    def __init__(self):
```

```

        self.books: Dict[int, Book] = {}
        self.members: Dict[int, Member] = {}
        self.borrow_log: List[Dict] = []
        self.genre_counter = Counter()

# ----- Book Management -----

def add_book(self, book_id: int, title: str, author: str, genre: str):
    """Add a new book to the library."""
    print(f"Adding book ID {book_id}...")
    if book_id in self.books:
        print("✗ Book ID already exists.")
        return

    self.books[book_id] = Book(book_id, title, author, genre)
    print("✓ Book added successfully.")

def search_books(self, keyword: str):
    """Search books by title or author."""
    print(f"Searching for books with keyword '{keyword}'...")
    return [
        book for book in self.books.values()
        if keyword.lower() in book.title.lower()
        or keyword.lower() in book.author.lower()
    ]

def get_available_books_by_genre(self, genre: str):
    """Return all available books in a given genre."""
    print(f"Fetching available books in genre '{genre}'...")
    return [
        book for book in self.books.values()
        if book.genre.lower() == genre.lower() and book.is_available
    ]

# ----- Member Management -----

def add_member(self, member_id: int, name: str, age: int, contact: str):
    """Add a new member to the library."""
    print(f"Adding member ID {member_id}...")
    if member_id in self.members:
        print("✗ Member ID already exists.")
        return

    self.members[member_id] = Member(member_id, name, age, contact)
    print("✓ Member added successfully.")

def get_members_with_borrowed_books(self):
    """Return list of members who have borrowed books."""
    print("Fetching members with borrowed books...")
    return [

```

```

        member for member in self.members.values()
        if member.borrowed_books
    ]

# ----- Borrow & Return System -----

def issue_book(self, member_id: int, book_id: int):
    """Issue a book to a member."""
    print(f"Issuing Book ID {book_id} to Member ID {member_id}...")

    if member_id not in self.members:
        print("✗ Invalid member ID.")
        return

    if book_id not in self.books:
        print("✗ Invalid book ID.")
        return

    book = self.books[book_id]
    member = self.members[member_id]

    if not book.is_available:
        print("✗ Book is already issued.")
        return

    print("Before Issue:", book, member.borrowed_books)

    book.is_available = False
    member.borrowed_books.append(book_id)

    self.borrow_log.append({
        "member_id": member_id,
        "book_id": book_id,
        "action": "borrow"
    })

    self.genre_counter[book.genre] += 1

    print("After Issue:", book, member.borrowed_books)
    print("☑ Book issued successfully.")

def return_book(self, member_id: int, book_id: int):
    """Return a book from a member."""
    print(f"Returning Book ID {book_id} from Member ID {member_id}...")

    if member_id not in self.members:
        print("✗ Invalid member ID.")
        return

    if book_id not in self.books:

```

```

        print("✗ Invalid book ID.")
        return

    book = self.books[book_id]
    member = self.members[member_id]

    if book_id not in member.borrowed_books:
        print("✗ This member did not borrow this book.")
        return

    print("Before Return:", book, member.borrowed_books)

    book.is_available = True
    member.borrowed_books.remove(book_id)

    self.borrow_log.append({
        "member_id": member_id,
        "book_id": book_id,
        "action": "return"
    })

    print("After Return:", book, member.borrowed_books)
    print("✅ Book returned successfully.")

# ----- Reports -----

def get_most_popular_genre(self):
    """Return the most popular genre."""
    print("Calculating most popular genre...")
    if not self.genre_counter:
        return None
    return self.genre_counter.most_common(1)[0][0]

# ===== TEST FUNCTIONS =====

def test_add_book():
    print("\n--- TEST: Add Book ---")
    library = Library()
    library.add_book(1, "Python Basics", "John Doe", "Programming")
    print(library.books)

def test_add_member():
    print("\n--- TEST: Add Member ---")
    library = Library()
    library.add_member(101, "Alice", 25, "email")
    print(library.members)

```

```
def test_issue_book():
    print("\n--- TEST: Issue Book ---")
    library = Library()

    library.add_book(1, "Python Basics", "John Doe", "Programming")
    library.add_member(101, "Alice", 25, "email")

    library.issue_book(101, 1)

def test_return_book():
    print("\n--- TEST: Return Book ---")
    library = Library()

    library.add_book(1, "Python Basics", "John Doe", "Programming")
    library.add_member(101, "Alice", 25, "email")
    library.issue_book(101, 1)

    library.return_book(101, 1)

def test_search_books():
    print("\n--- TEST: Search Books ---")
    library = Library()

    library.add_book(1, "Python Basics", "John Doe", "Programming")
    library.add_book(2, "Advanced Python", "Jane Smith", "Programming")

    results = library.search_books("Python")
    print(results)

def test_available_books_by_genre():
    print("\n--- TEST: Available Books by Genre ---")
    library = Library()

    library.add_book(1, "Python Basics", "John Doe", "Programming")
    library.add_book(2, "AI Book", "Jane", "AI")

    library.add_member(101, "Alice", 25, "email")
    library.issue_book(101, 1)

    print(library.get_available_books_by_genre("Programming"))

def test_members_with_borrowed_books():
    print("\n--- TEST: Members with Borrowed Books ---")
    library = Library()

    library.add_book(1, "Python", "John", "Programming")
```

```
library.add_member(101, "Alice", 25, "email")

library.issue_book(101, 1)

print(library.get_members_with_borrowed_books())

def test_most_popular_genre():
    print("\n--- TEST: Most Popular Genre ---")
    library = Library()

    library.add_book(1, "Python", "John", "Programming")
    library.add_book(2, "AI", "Jane", "AI")

    library.add_member(101, "Alice", 25, "email")

    library.issue_book(101, 1)
    library.issue_book(101, 2)

    print(library.get_most_popular_genre())

# ===== RUN ALL TESTS =====

if __name__ == "__main__":
    test_add_book()
    test_add_member()
    test_issue_book()
    test_return_book()
    test_search_books()
    test_available_books_by_genre()
    test_members_with_borrowed_books()
    test_most_popular_genre()
```

Technical Notes (Design Decisions & Assumptions)

This system is designed using an **Object-Oriented Programming (OOP)** approach to ensure modularity, readability, and scalability. Core entities—`Book`, `Member`, and `Library`—are modeled as separate classes to maintain **separation of concerns**. The `Library` class acts as the central controller managing all operations, including book handling, member management, and transactions.

Data is stored using in-memory structures:

- `dict` for fast lookup of books and members by ID ($O(1)$ access)
- `list` for borrow logs and member book tracking
- `Counter` for efficient genre popularity tracking

Each function is designed to be **reusable and independently testable**, supported by debug print statements to trace execution flow (before/after state changes). Validation checks (e.g., preventing issuing an already borrowed book) ensure logical consistency and error handling.

Key Design Decisions

- **Boolean flag** (`is_available`) used instead of string status for simplicity and efficiency.
- **Borrow log** maintained as a list of dictionaries to track all transactions without complex structures.
- **Genre popularity** tracked incrementally during issue operations instead of recalculating repeatedly.

Assumptions

- Each book copy is unique (no multiple copies of same book ID).
- No limit on number of books a member can borrow.
- No due dates, fines, or reservation system included.
- Data is not persisted (resets on program restart).
- Inputs are assumed to be valid data types (basic validation only).

This design prioritizes **clarity and extensibility**, making it easy to upgrade with features like databases, APIs, or UI layers.

Sample Runs

```
PS C:\study\AI\IITM_Agentic_AI_Training> & C:/Users/Ankur/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/st
cts/Week 4/Solution/library_management_system.py"
```

```
--- TEST: Add Book ---
Adding book ID 1...
✅ Book added successfully.
{1: [1] Python Basics by John Doe (Programming) - Available}

--- TEST: Add Member ---
Adding member ID 101...
✅ Member added successfully.
{101: [101] Alice, Age: 25, Contact: email}

--- TEST: Issue Book ---
Adding book ID 1...
✅ Book added successfully.
Adding member ID 101...
✅ Member added successfully.
Issuing Book ID 1 to Member ID 101...
Before Issue: [1] Python Basics by John Doe (Programming) - Available []
After Issue: [1] Python Basics by John Doe (Programming) - Issued [1]
✅ Book issued successfully.

--- TEST: Return Book ---
Adding book ID 1...
✅ Book added successfully.
Adding member ID 101...
✅ Member added successfully.
Issuing Book ID 1 to Member ID 101...
Before Issue: [1] Python Basics by John Doe (Programming) - Available []
After Issue: [1] Python Basics by John Doe (Programming) - Issued [1]
✅ Book issued successfully.
Returning Book ID 1 from Member ID 101...
Before Return: [1] Python Basics by John Doe (Programming) - Issued [1]
After Return: [1] Python Basics by John Doe (Programming) - Available []
✅ Book returned successfully.
```

```
--- TEST: Search Books ---
Adding book ID 1...
✅ Book added successfully.
Adding book ID 2...
✅ Book added successfully.
Searching for books with keyword 'Python'...
[[1] Python Basics by John Doe (Programming) - Available, [2] Advanced Python by Jane Smith (Programming) - Available]

--- TEST: Available Books by Genre ---
Adding book ID 1...
✅ Book added successfully.
Adding book ID 2...
✅ Book added successfully.
Adding member ID 101...
✅ Member added successfully.
Issuing Book ID 1 to Member ID 101...
Before Issue: [1] Python Basics by John Doe (Programming) - Available []
After Issue: [1] Python Basics by John Doe (Programming) - Issued [1]
✅ Book issued successfully.
Fetching available books in genre 'Programming'...
[]

--- TEST: Members with Borrowed Books ---
Adding book ID 1...
✅ Book added successfully.
Adding member ID 101...
✅ Member added successfully.
Issuing Book ID 1 to Member ID 101...
Before Issue: [1] Python by John (Programming) - Available []
After Issue: [1] Python by John (Programming) - Issued [1]
✅ Book issued successfully.
Fetching members with borrowed books...
[[101] Alice, Age: 25, Contact: email]
```


--- TEST: Most Popular Genre ---

C:\study\AI\IITM_Agentic_AI_Training\Live Session Activity

✔ Book added successfully.

Adding book ID 2...

✔ Book added successfully.

Adding member ID 101...

✔ Member added successfully.

Issuing Book ID 1 to Member ID 101...

Before Issue: [1] Python by John (Programming) - Available []

After Issue: [1] Python by John (Programming) - Issued [1]

✔ Book issued successfully.

Issuing Book ID 2 to Member ID 101...

Before Issue: [2] AI by Jane (AI) - Available [1]

After Issue: [2] AI by Jane (AI) - Issued [1, 2]

✔ Book issued successfully.

Calculating most popular genre...

Programming

PS C:\study\AI\IITM_Agentic_AI_Training> []